

Secure vs Insecure Login System (PHP + MySQL)

Prepared by - Mubasshera Zaidy

Introduction

Web applications often rely on login systems to authenticate users. However, insecure coding practices can expose these systems to attacks such as SQL Injection, allowing unauthorized access. This project demonstrates both an insecure login implementation (vulnerable to SQL Injection) and a secure login implementation (protected using prepared statements and secure coding practices). By comparing the two, we highlight the importance of secure development in preventing exploitation.

Objectives

- To design and implement a basic login system using Python with SQLite/MySQL.
- To demonstrate how SQL Injection can bypass authentication in insecure systems.
- To implement secure coding practices (prepared statements) that prevent SQL Injection.
- To evaluate the difference between insecure and secure login systems.

Scope

This project focuses on demonstrating login system security using a simple web application connected to a database. It covers two implementations:

- An insecure login vulnerable to SQL Injection.
- A secure login using prepared statements.

The scope is limited to authentication functionality and does not extend to advanced features such as password hashing, multi-factor authentication, or session management.

Methodology

1. Created HTML Login Pages

- Designed two login forms: one for insecure login and one for secure login.
- Each form collected username and password inputs.

Secure Login HTML page :

```
1  <!DOCTYPE html>
2  <html>
3  <head><title>Secure Login</title></head>
4  <body>
5  <h2>Secure Login (SQL Injection prevented)</h2>
6  <form action="secure_login.php" method="POST">
7      Username: <input type="text" name="username"><br><br>
8      Password: <input type="password" name="password"><br><br>
9      <input type="submit" value="Login">
10 </form>
11 </body>
12 </html>
```

Insecure Login HTML page :

```
1  <!DOCTYPE html>
2  <html>
3  <head><title>Insecure Login</title></head>
4  <body>
5  <h2>Insecure Login (SQL Injection possible)</h2>
6  <form action="insecure_login.php" method="POST">
7      Username: <input type="text" name="username"><br><br>
8      Password: <input type="password" name="password"><br><br>
9      <input type="submit" value="Login">
10 </form>
11 </body>
12 </html>
13
```

2. Developed PHP Backend Scripts

- **Insecure Version:** Used direct string concatenation in SQL queries, making it vulnerable to SQL Injection.

```
1  <?php
2  $conn = new mysqli("localhost", "root", "", "testdb");
3
4  $username = $_POST['username'];
5  $password = $_POST['password'];
6
7  // Vulnerable query: direct concatenation
8  $sql = "SELECT * FROM users WHERE username='$username' AND password='$password'";
9  $result = $conn->query($sql);
10
11 if ($result->num_rows > 0) {
12     echo "Login successful (UNSECURE)!";
13 } else {
14     echo "Login failed.";
15 }
16 ?>
17
```

- **Secure Version:** Used prepared statements (bind_param) to safely handle user input.

```
1  <?php
2  $conn = new mysqli("localhost", "root", "", "testdb");
3
4  $username = $_POST['username'];
5  $password = $_POST['password'];
6
7  // Secure query: prepared statement
8  $stmt = $conn->prepare("SELECT * FROM users WHERE username=? AND password=?");
9  $stmt->bind_param("ss", $username, $password);
10 $stmt->execute();
11 $result = $stmt->get_result();
12
13 if ($result->num_rows > 0) {
14     echo "Login successful (SECURE)!";
15 } else {
16     echo "Login failed.";
17 }
18 ?>
```

3. Database Setup

- Created a database named testdb in MySQL.
- Added a users table with fields for id, username and password
- Inserted sample user credentials (e.g., admin, admin123).

Server: localhost » Database: testdb » Table: users

[Browse](#) [Structure](#) [SQL](#) [Search](#) [Insert](#) [Export](#) [Import](#)

✓ Showing rows 0 - 0 (1 total, Query took 0.0001 seconds.)

```
SELECT * FROM `users`
```

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 Filter rows:

Extra options

↩ T ↪

▼

id

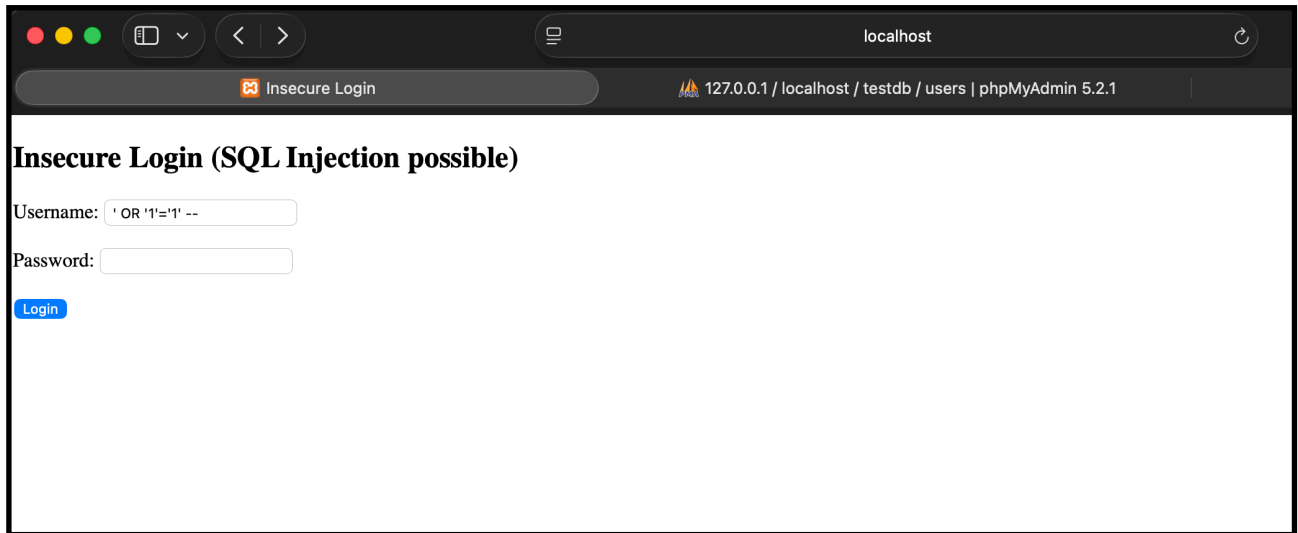
username

password

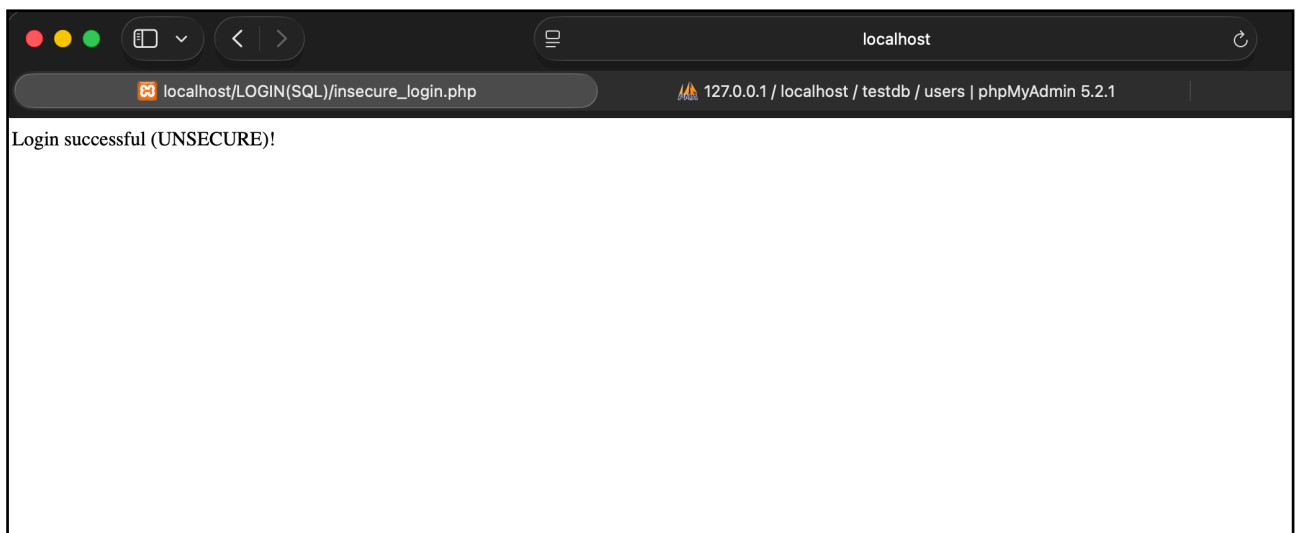
☐ Edit Copy Delete 1 admin admin123

4. Testing Insecure Login

- Attempted SQL Injection payloads such as ' OR '1'='1' -- in the username field.

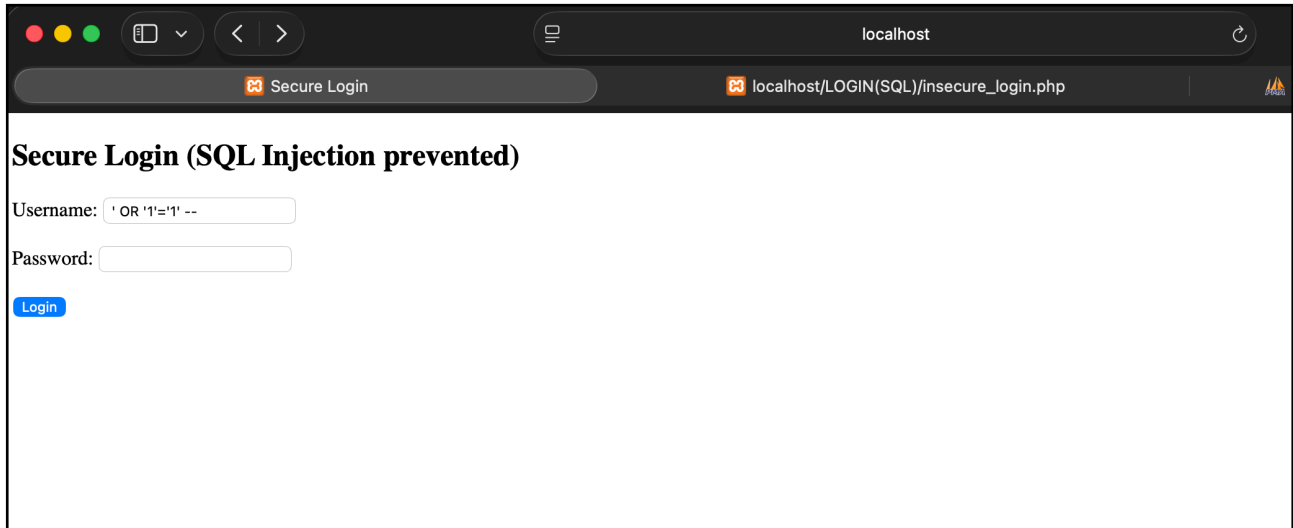


- Result: Login was bypassed successfully, proving the vulnerability.

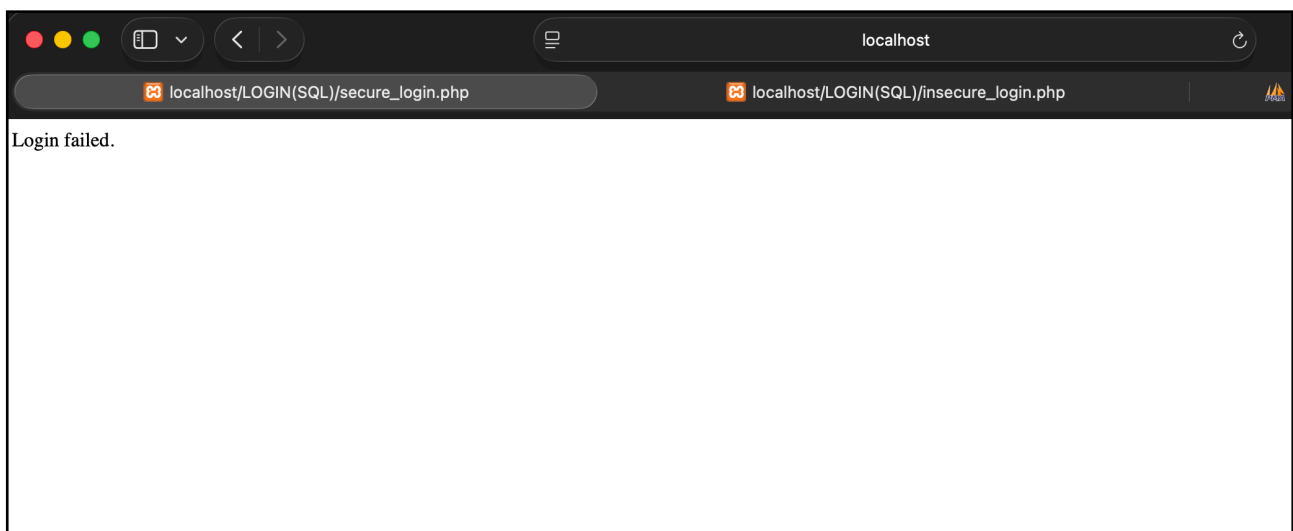


5. Testing Secure Login

- Used the same SQL Injection payloads on the secure login page.



- Result: Login failed, as prepared statements treated the input as plain text, preventing injection.



Recommendations

- Always use **prepared statements** or **parameterized queries** to prevent SQL Injection.
- Store passwords securely using **hashing algorithms** (e.g., bcrypt, password_hash() in PHP).
- Implement **role-based access control** to limit privileges.
- Use **HTTPS** to protect credentials during transmission.
- Regularly audit and test applications for vulnerabilities.

Conclusion

This project successfully demonstrated the difference between insecure and secure login systems. The insecure version allowed SQL Injection attacks to bypass authentication, while the secure version blocked such attempts using prepared statements. The exercise highlights the critical importance of secure coding practices in web application development. By adopting these practices, developers can significantly reduce the risk of unauthorized access and protect sensitive user data.